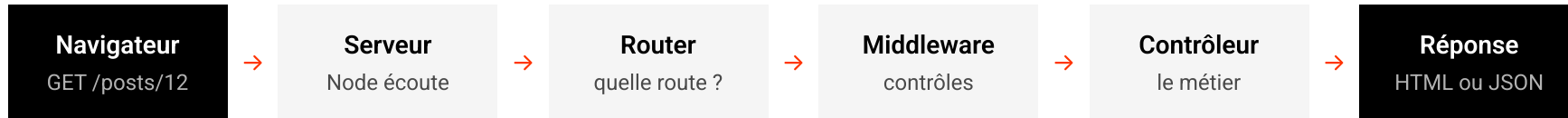


Routing & contrôleurs

Le voyage d'une requête

Ce qui se passe entre le clic et l'affichage



Une requête, c'est du texte. Un verbe (GET , POST ...), un chemin, des en-têtes, parfois un corps. Rien de magique : vous pourriez l'écrire à la main.

Une réponse aussi. Un code de statut (200, 404, 500), des en-têtes, un corps. Le navigateur ne fait qu'interpréter ce texte.

Déclarer des routes

`start/routes.ts` : la table des matières de votre application

```
import router from '@adonisjs/core/services/router'

router.get('/', () => 'Bonjour')

// Un paramètre dynamique
router.get('/posts/:id', ({ params }) => {
  return `Article numéro ${params.id}`
})

// Paramètre optionnel
router.get('/posts/:id?', ({ params }) => {
  return params.id ? `Article ${params.id}` : 'Tous les articles'
})

// Le verbe fait partie de la route : même chemin, intention différente
router.post('/posts', () => 'Créer un article')
router.put('/posts/:id', () => 'Modifier un article')
router.delete('/posts/:id', () => 'Supprimer un article')
```

`node ace list:routes` affiche toutes les routes de l'application. Le premier réflexe sur un projet inconnu.

Des routes aux contrôleurs

Quand la logique dépasse trois lignes, elle sort du fichier de routes

Créer un contrôleur

```
node ace make:controller posts
```

Le brancher

```
// start/routes.ts
import router from '@adonisjs/core/services/router'
import { controllers } from '#generated/controllers'

router.get('/posts', [controllers.Posts, 'index'])
router.get('/posts/:id', [controllers.Posts, 'show'])
router.post('/posts', [controllers.Posts, 'store'])
```

Le contrôleur

```
// app/controllers/posts_controller.ts
import type { HttpContext } from '@adonisjs/core/http'

export default class PostsController {
  async index({ view }: HttpContext) {
    return view.render('posts/index')
  }

  async show({ params }: HttpContext) {
    return `Article ${params.id}`
  }

  async store({ request }: HttpContext) {
    const data = request.all()
    return data
  }
}
```

Sept routes en une ligne

La même structure revient pour chaque ressource, autant la déclarer d'un coup

```
// start/routes.ts
router.resource('movies', controllers.Movies)
```

Équivaut à écrire les sept routes à la main. Et pour générer le contrôleur qui va avec :

```
node ace make:controller movies --resource
```

On ne prend que ce dont on a besoin

```
// Sans les formulaires, pour une API
router.resource('movies', controllers.Movies).apiOnly()

// À la carte
router.resource('movies', controllers.Movies)
  .only(['index', 'show'])

// Ressource imbriquée : /movies/:movie_id/reviews
router.resource('movies.reviews', controllers.Reviews)
```

Les sept actions

GET /movies

→ index • la liste

GET /movies/create

→ create • le formulaire

POST /movies

→ store • l'enregistrement

GET /movies/:id

→ show • la fiche

GET /movies/:id/edit

→ edit • le formulaire

PUT /movies/:id

→ update • la modification

Le contexte HTTP

Un seul objet contient tout ce que vous savez de la requête en cours

Lire

request · le corps, les en-têtes, la méthode

params · les morceaux dynamiques de l'URL

auth · l'utilisateur connecté, s'il y en a un

session · ce qu'on garde d'une requête à l'autre

Répondre

response · statut, en-têtes, redirection

view · rendre un template Edge

inertia · rendre une page Vue ou React

bouncer · vérifier les droits

Ce contexte est **créé pour chaque requête et détruit ensuite**. Deux utilisateurs simultanés ont deux contextes distincts : rien ne fuit de l'un à l'autre. C'est ce qui rend le serveur capable de répondre à plusieurs personnes à la fois sans les confondre.

Travaux dirigés

Le ciné-club : les premières routes

TD : Poser la carte du site

Aucune base de données aujourd'hui : on dessine la circulation, on branche les contrôleurs

1. Les routes à déclarer

```
// start/routes.ts
router.get('/movies', [controllers.Movies, 'index'])
router.get('/movies/:id', [controllers.Movies, 'show'])
router.get('/movies/:id/reviews', [controllers.Reviews, 'show'])
router.post('/movies/:id/reviews', [controllers.Reviews, 'create'])
```

2. Les contrôleurs

```
node ace make:controller movies -r
node ace make:controller reviews -r
```

3. Vérifier

```
node ace list:routes
```

4. Renvoyer des données en dur

Pas encore de base : le contrôleur renvoie un tableau écrit à la main. L'objectif est de voir circuler l'information, pas de la stocker.

À faire vivre : déclarez `/movies/new` **après** `/movies/:id`, puis visitez l'URL. Observez ce que reçoit le contrôleur, et corrigez.

Pour aller plus loin : remplacez vos routes par un `router.resource()`, puis comparez `list:routes` avant et après. Nommez aussi vos routes et générez leurs URL plutôt que de les écrire en dur.